Custom Annotations and AOP Binding in Spring

Spring AOP becomes especially useful when a method can declare **what behaviour it needs**, while an aspect contains **how that behaviour is implemented**.

A custom annotation gives us a clean, declarative way to connect business methods with cross-cutting concerns such as:

- Execution-time tracking
- Logging
- Auditing
- Authorization
- Rate limiting
- Retry handling
- Validation

---

1. Separating “What” from “How”

Consider a service method that measures its own execution time:

```
public Report generateAnnualReport() {

    long startTime = System.nanoTime();

    try {
        return reportGenerator.generate();
    } finally {
        long duration = System.nanoTime() - startTime;
        System.out.println("Duration: " + duration);
```

```
    }
}
```

The method now has two responsibilities:

1. Generating the report
2. Measuring execution time

Execution-time measurement is not part of the core business logic. It is a **cross-cutting concern** that may be required by many methods.

We can move that behaviour into an aspect:

```
@Around("@annotation(TrackExecutionTime)")
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint
) throws Throwable {
    // Execution-time logic
}
```

The business method only declares its intention:

```
@TrackExecutionTime
public Report generateAnnualReport() {
    return reportGenerator.generate();
}
```

The responsibilities are now separated:

```
Annotation
    → Declares what behaviour is required

Aspect
    → Defines how that behaviour is implemented

Business method
    → Contains the actual business logic
```

This is an example of **declarative programming**. The method declares the behaviour it needs without implementing that behaviour itself.

---

2. A Custom Annotation Does Nothing by Itself

Suppose we create an annotation:

```
public @interface TrackExecutionTime {
}
```

and apply it to a method:

```
@TrackExecutionTime
public void generateReport() {
}
```

Nothing happens automatically.

An annotation is only **metadata** attached to a program element.

```
Method
  +
Metadata: “Track the execution time of this method”
```

Some other component must:

1. Detect the annotation
2. Read its information
3. Perform the required behaviour

In this case, the Spring AOP aspect performs that work:

```
@Around("@annotation(TrackExecutionTime)")
public Object trackTime(
        ProceedingJoinPoint joinPoint
```

```
) throws Throwable {
    // Timing implementation
}
```

> The annotation declares the intention. The aspect gives that annotation its behaviour.

---

3. Creating a Custom Annotation

A custom annotation is declared using `@interface`:

```
public @interface TrackExecutionTime {
}
```

It is not declared using the normal `interface` keyword.

For Spring AOP, the basic declaration is usually not enough. Java must also know:

- Where the annotation can be used
- How long the annotation remains available

These rules are defined through **meta-annotations**.

A meta-annotation is an annotation applied to another annotation declaration.

The most important meta-annotations for custom AOP annotations are:

```
@Target
@Retention
```

`@Documented` is also commonly added.

---

4. Understanding `@Target`

`@Target` defines where an annotation is legally allowed to appear.

For an annotation that should be placed on methods:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
public @interface TrackExecutionTime {
}
```

The compiler now permits this:

```
@TrackExecutionTime
public void generateReport() {
}
```

but does not permit the annotation on a class:

```
@TrackExecutionTime // Compilation error with METHOD-only tar
get
public class ReportService {
}
```

Common `ElementType` values

`ElementType.METHOD`

The annotation can be applied to methods.

```
@Target(ElementType.METHOD)
```

`ElementType.TYPE`

The annotation can be applied to classes, interfaces, enums, records and annotation interfaces.

```
@Target(ElementType.TYPE)
```

`ElementType.FIELD`

The annotation can be applied to fields.

```
@Target(ElementType.FIELD)
```

`ElementType.PARAMETER`

The annotation can be applied to method or constructor parameters.

```
@Target(ElementType.PARAMETER)
```

`ElementType.CONSTRUCTOR`

The annotation can be applied to constructors.

```
@Target(ElementType.CONSTRUCTOR)
```

`ElementType.ANNOTATION_TYPE`

The annotation can be applied to another annotation declaration.

```
@Target(ElementType.ANNOTATION_TYPE)
```

For method-level Spring AOP, the most common choice is:

```
@Target(ElementType.METHOD)
```

Spring AOP is proxy-based and primarily intercepts method executions on Spring-managed beans.

Supporting classes and methods

An annotation may support more than one target:

```
@Target({
    ElementType.TYPE,
```

```
    ElementType.METHOD
})
```

This allows the annotation to be placed on either a class or an individual method.

---

5. Understanding `@Retention`

`@Retention` defines how long annotation information is preserved.

Java provides three retention policies:

```
RetentionPolicy.SOURCE
RetentionPolicy.CLASS
RetentionPolicy.RUNTIME
```

`RetentionPolicy.SOURCE`

```
@Retention(RetentionPolicy.SOURCE)
```

The annotation exists only in the source code and is discarded during compilation.

It is useful for compiler-related tools but cannot be inspected by Spring while the application is running.

`RetentionPolicy.CLASS`

```
@Retention(RetentionPolicy.CLASS)
```

The annotation is stored in the compiled `.class` file but is not normally available through runtime reflection.

`CLASS` is Java's default retention policy when `@Retention` is not specified.

`RetentionPolicy.RUNTIME`

```
@Retention(RetentionPolicy.RUNTIME)
```

The annotation remains available while the application is running.

Spring and Java reflection can therefore inspect it at runtime.

Custom annotations used for Spring AOP should normally use:

```
@Retention(RetentionPolicy.RUNTIME)
```

Without runtime retention, Spring cannot reliably detect and bind the annotation while executing the application.

---

6. Understanding `@Documented`

`@Documented` tells Java documentation tools to include the annotation in generated Javadoc.

```
@Documented
public @interface TrackExecutionTime {
}
```

It improves documentation but does not affect AOP matching or proxy creation.

A typical method-level AOP annotation therefore looks like this:

```
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface TrackExecutionTime {
}
```

---

7. Marker Annotations and Configured Annotations

A custom annotation can either act as a simple marker or carry additional configuration.

Marker annotation

A marker annotation has no properties:

```
public @interface TrackExecutionTime {
}
```

It communicates only one thing:

```
Apply execution-time tracking to this method.
```

Usage:

```
@TrackExecutionTime
public Report generateReport() {
    return reportGenerator.generate();
}
```

Annotation with properties

An annotation can also carry values:

```
public @interface TrackExecutionTime {

    String operation() default "";

    long warnAfterMillis() default 1000;
}
```

Usage:

```
@TrackExecutionTime(
    operation = "Generate annual report",
    warnAfterMillis = 500
)
public Report generateAnnualReport() {
    return reportGenerator.generate();
}
```

The method now declares:

- That execution time should be measured
- The name of the operation
- The threshold after which the operation should be treated as slow

The aspect can read these values:

```
trackExecutionTime.operation()
```

```
trackExecutionTime.warnAfterMillis()
```

This makes the annotation both a marker and a small configuration object.

---

8. The `@annotation()` Pointcut Designator

The `@annotation()` pointcut designator matches methods based on an annotation present on the method being executed.

It can be used in two important ways:

1. Type matching
2. Annotation binding

8.1 Type-Matching Form

```
@annotation(com.coderarmy.student.aop.annotation.TrackExecuti
onTime)
```

This asks:

> Is the executed method annotated with `TrackExecutionTime`?

Use this form when the advice only needs to know whether the annotation exists.

```
@Before(
    "@annotation(com.coderarmy.student.aop.annotation.TrackEx
ecutionTime)"
)
public void beforeTrackedMethod() {
    System.out.println("Tracked method called");
}
```

The fully qualified annotation name is written directly inside the pointcut expression.

8.2 Binding Form

```
@annotation(trackExecutionTime)
```

This form performs two jobs:

1. It matches methods containing the annotation
2. It passes the actual annotation instance to the advice method

```
@Before("@annotation(trackExecutionTime)")
public void beforeTrackedMethod(
        TrackExecutionTime trackExecutionTime
) {
    System.out.println(
        trackExecutionTime.operation()
```

```
    );
}
```

The name used in the pointcut expression corresponds to the advice parameter:

```
@annotation(trackExecutionTime)
            └─────────────────┘

TrackExecutionTime trackExecutionTime
                   └─────────────────┘
```

Binding is required when the aspect needs to read annotation properties such as:

```
operation()
warnAfterMillis()
```

> Type matching checks whether the annotation is present. Binding also gives the advice access to the annotation object.

---

9. Complete Example: Measuring Execution Time

Step 1: Create the annotation

```
package com.coderarmy.student.aop.annotation;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
```

```
@Documented
public @interface TrackExecutionTime {

    String operation() default "";

    long warnAfterMillis() default 1000;
}
```

Step 2: Apply it to a service method

```
package com.coderarmy.student.service;

import com.coderarmy.student.aop.annotation.TrackExecutionTim
e;
import org.springframework.stereotype.Service;

@Service
public class ReportService {

    @TrackExecutionTime(
        operation = "Generate annual student report",
        warnAfterMillis = 500
    )
    public StudentReport generateAnnualReport() {
        return createReport();
    }

    private StudentReport createReport() {
        return new StudentReport();
    }
}
```

The service method contains only business logic. It does not contain timing, logging or threshold-handling code.

Step 3: Create the aspect

```
package com.coderarmy.student.aop.aspect;

import com.coderarmy.student.aop.annotation.TrackExecutionTim
e;
import java.util.concurrent.TimeUnit;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class PerformanceAspect {

    @Around("@annotation(trackExecutionTime)")
    public Object measureExecutionTime(
            ProceedingJoinPoint joinPoint,
            TrackExecutionTime trackExecutionTime
    ) throws Throwable {

        long startTime = System.nanoTime();

        try {
            return joinPoint.proceed();
        } finally {
            long durationNanos =
                    System.nanoTime() - startTime;

            long durationMillis =
                    TimeUnit.NANOSECONDS.toMillis(
                            durationNanos
                    );
```

```
            String operation =
                    trackExecutionTime.operation();

            if (operation.isBlank()) {
                operation =
                        joinPoint.getSignature()
                                .toShortString();
            }

            long warningThreshold =
                    trackExecutionTime
                            .warnAfterMillis();

            if (durationMillis >= warningThreshold) {
                System.out.println(
                        "SLOW OPERATION: "
                        + operation
                        + " took "
                        + durationMillis
                        + " ms"
                );
            } else {
                System.out.println(
                        operation
                        + " took "
                        + durationMillis
                        + " ms"
                );
            }
        }
    }
}
```

Execution flow

When another Spring bean calls `generateAnnualReport()`:

```
Caller
   ↓
AOP proxy
   ↓
PerformanceAspect.measureExecutionTime()
   ↓
joinPoint.proceed()
   ↓
ReportService.generateAnnualReport()
   ↓
Result returns to the aspect
   ↓
Aspect calculates and logs the duration
   ↓
Result returns to the caller
```

The `finally` block ensures that duration is calculated whether the target method:

- Returns successfully
- Throws an exception

The original exception is not swallowed because the advice does not catch and replace it.

---

10. Where Spring AOP Enters the Bean Lifecycle

At a simplified level, Spring creates a bean through the following stages:

```
1. Read the bean definition
2. Instantiate the bean
3. Inject dependencies
4. Run Aware callbacks
5. Run BeanPostProcessors before initialization
6. Run initialization callbacks
7. Run BeanPostProcessors after initialization
8. Store and expose the final bean reference
```

Spring AOP's automatic proxy creation is built on Spring's bean post-processing infrastructure.

An AOP auto-proxy creator examines beans during their creation and determines whether any configured advice applies to them.

Conceptually:

```
Spring creates the StudentService target
                    ↓
Injects its dependencies
                    ↓
Runs initialization callbacks
                    ↓
AOP infrastructure examines the bean
                    ↓
Does any advisor match this bean?
           ↙                    ↘
         No                      Yes
         ↓                        ↓
Return original bean       Create AOP proxy
                                  ↓
                           Return the proxy
```

For an eligible bean, the final reference exposed by the container is normally:

```
Proxy → Target
```

rather than only:

```
Target
```

> AOP advice does not run while the bean is merely being created. Spring first creates the proxy. Advice runs later when a method is invoked through that proxy.

---

11. BeanPostProcessor from First Principles

`BeanPostProcessor` is a Spring extension point that allows infrastructure code to inspect or modify beans created by the container.

Its basic idea is:

> Whenever Spring creates a bean, infrastructure components get an opportunity to process it before the bean is finally exposed.

A simplified form of the interface is:

```
public interface BeanPostProcessor {

    Object postProcessBeforeInitialization(
            Object bean,
            String beanName
    );

    Object postProcessAfterInitialization(
            Object bean,
            String beanName
    );
}
```

A post-processor may return the original bean:

```
return bean;
```

or return a wrapped object:

```
return proxy;
```

`postProcessBeforeInitialization()`

This method runs before initialization callbacks such as:

- `@PostConstruct`
- `InitializingBean.afterPropertiesSet()`

- A custom `initMethod`

It may be used to:

- Inspect the bean
- Perform additional configuration
- Process certain annotations
- Validate or prepare the bean before initialization

`postProcessAfterInitialization()`

This method runs after initialization callbacks have completed.

Spring's AOP auto-proxy creation normally wraps eligible beans around this stage and returns the proxy as the final bean reference.

```
Original bean created
        ↓
Initialization completed
        ↓
AOP post-processing
        ↓
Original bean or proxy returned
```

The exact internal implementation is more advanced, but the important conceptual point is:

> Spring AOP uses bean post-processing to replace an eligible bean reference with a proxy reference.

---

12. How Spring Decides Which Beans Need Proxies

Consider the following aspect:

```
@Aspect
@Component
```

```java
public class LoggingAspect {

    @Before(
        "within(com.coderarmy.student.service..*)"
    )
    public void log() {
        System.out.println("Service method called");
    }
}
```

Spring processes the aspect and creates an internal representation of:

```
Advice
    → Run log()

Pointcut
    → Match classes inside the service package
```

The combination of a pointcut and advice is commonly represented internally as an **advisor**.

As Spring creates beans, the AOP infrastructure checks whether the advisors can apply to each bean.

For `StudentController`:

```
Is StudentController inside the service package?
No
```

No proxy is required for this aspect.

For `StudentService`:

```
Is StudentService inside the service package?
Yes
```

Spring creates an AOP proxy for the service bean.

The same idea applies to an annotation-based pointcut:

```
@Around("@annotation(trackExecutionTime)")
```

Spring checks whether the bean contains methods that can match the annotation-based advisor. Eligible calls are intercepted by the proxy at runtime.

---

13. The Container Stores the Final Reference

Suppose Spring first creates the target object:

```
Target object: T100
```

It then creates a proxy:

```
Proxy object: P200
```

The relationship becomes:

```
P200: Proxy
   ↓
T100: Target
```

When another bean asks Spring for `StudentService`, the container supplies:

```
P200
```

not the raw target reference:

```
T100
```

For example:

```
public StudentController(
        StudentService studentService
) {
    this.studentService = studentService;
}
```

Conceptually, the field stores the proxy:

```
StudentController
studentService field
       |
       ▼
    AOP proxy
       |
       ▼
  StudentService target
```

This is why dependency injection and AOP work together.

Spring controls:

- Object creation
- Dependency injection
- Bean post-processing
- The final reference supplied to other beans

Therefore, Spring can inject the proxy instead of the raw object.

---

14. The Method Interceptor Chain

A single method may match multiple cross-cutting concerns.

For example:

```
Logging
Security
Performance measurement
```

The runtime chain may look like this:

```
Caller
   ↓
Proxy
   ↓
Logging interceptor
   ↓
Security interceptor
   ↓
Performance interceptor
   ↓
Target method
```

When the target method completes, execution unwinds in reverse order:

```
Target method returns
   ↓
Performance interceptor completes
   ↓
Security interceptor completes
   ↓
Logging interceptor completes
   ↓
Proxy returns the result
   ↓
Caller receives the result
```

This behaves like nested method calls:

```
Logging(
    Security(
        Performance(
            Target method
        )
    )
)
```

Each interceptor gets an opportunity to run code:

- Before continuing the chain
- After the chain returns

- When the chain throws an exception

15. What `proceed()` Actually Means

A common beginner interpretation is:

```
proceed() directly calls the target method
```

That is not always precise.

`proceed()` means:

> Continue to the next interceptor in the current chain.

If another interceptor still remains, that interceptor runs next.

Only the final `proceed()` in the chain reaches the target method.

```
First around advice
    ↓ proceed()
Second around advice
    ↓ proceed()
Third around advice
    ↓ proceed()
Target method
```

An `@Around` advice controls whether the chain continues.

If it does not call `proceed()`:

```
@Around("execution(* com.coderarmy..*(..))")
public Object intercept(
        ProceedingJoinPoint joinPoint
) {
    return "Blocked";
}
```

then:

- Remaining interceptors are not invoked
- The target method is not executed
- The advice directly returns its own value

Therefore, `@Around` is the most powerful advice type.

---

16. Controlling Aspect Order with `@Order`

When multiple aspects match the same method, `@Order` controls their precedence.

```
@Aspect
@Component
@Order(1)
public class SecurityAspect {
}
```

```
@Aspect
@Component
@Order(2)
public class LoggingAspect {
}
```

```
@Aspect
@Component
@Order(3)
public class PerformanceAspect {
}
```

The fundamental rule is:

> A lower order value has higher precedence.

For the example above, execution enters the aspects in this order:

```
SecurityAspect      @Order(1)
        ↓
LoggingAspect       @Order(2)
        ↓
PerformanceAspect   @Order(3)
        ↓
Target method
```

After the target method completes, execution returns in reverse order:

```
Target method
        ↓
PerformanceAspect completes
        ↓
LoggingAspect completes
        ↓
SecurityAspect completes
```

Another way to visualize it is:

```
Security(
    Logging(
        Performance(
            Target method
        )
    )
)
```

The lower order value forms the outer layer of the interceptor chain.

---

17. Important Proxy-Based AOP Boundaries

Custom annotation-based AOP works only when the method call passes through the Spring AOP proxy.

The object must be managed by Spring

A manually created object is not automatically proxied:

```
ReportService service = new ReportService();
```

Calling an annotated method on this object bypasses Spring AOP.

The object should be obtained from the Spring container through dependency injection.

Self-invocation bypasses the proxy

Consider:

```
@Service
public class ReportService {

    public void generateAllReports() {
        generateAnnualReport();
    }

    @TrackExecutionTime
    public void generateAnnualReport() {
    }
}
```

When `generateAllReports()` calls `generateAnnualReport()` using `this`, the call stays inside the target object:

```
Target method
   ↓
this.generateAnnualReport()
```

It does not return to the proxy, so the annotation-based advice is not applied to that internal call.

Private methods are not suitable join points for Spring proxy-based AOP

Spring AOP intercepts method calls through proxies. Private methods cannot be overridden or exposed through a proxy in the required way.

Custom AOP annotations should therefore normally be applied to externally invoked, non-private methods on Spring-managed beans.

18. Complete Mental Model

During application startup

```
Spring reads bean definitions
        ↓
Creates target objects
        ↓
Injects dependencies
        ↓
Runs initialization callbacks
        ↓
AOP infrastructure checks available advisors
        ↓
Creates proxies for eligible beans
        ↓
Stores and injects the final proxy references
```

During method execution

```
Caller invokes a method
        ↓
Call reaches the proxy
        ↓
Proxy finds matching interceptors
        ↓
Interceptor chain begins
        ↓
Around advice calls proceed()
        ↓
Target method executes
        ↓
Chain unwinds in reverse order
        ↓
Result or exception returns to the caller
```

Responsibility of each part

```
Custom annotation
    → Declares the required behaviour

Pointcut expression
    → Selects methods carrying that annotation

Advice
    → Implements the cross-cutting behaviour

AOP proxy
    → Intercepts method calls

BeanPostProcessor infrastructure
    → Creates and exposes the proxy during bean creation

ProceedingJoinPoint.proceed()
    → Continues the interceptor chain
```

---

19. Key Takeaways

- A custom annotation is metadata; it has no behaviour by itself.
- `@Target` controls where an annotation may be used.
- `@Retention(RUNTIME)` keeps the annotation available for Spring at runtime.
- `@Documented` affects generated documentation, not AOP behaviour.
- A marker annotation only declares that behaviour is required.
- Annotation properties can provide configuration to an aspect.
- `@annotation(TypeName)` matches an annotation by type.
- `@annotation(variableName)` binds the annotation instance to an advice parameter.
- Spring AOP uses bean post-processing infrastructure to create proxies for eligible beans.
- The Spring container stores and injects the final proxy reference.
- A method call must pass through the proxy for advice to execute.

- `proceed()` continues the interceptor chain; it does not always call the target immediately.
- Multiple interceptors execute as a nested chain and unwind in reverse order.
- A lower `@Order` value gives an aspect higher precedence.
- Self-invocation and private methods are important limitations of proxy-based Spring AOP.